GATE DA 2027 – Missing Topics Addendum

Ramu and Somu

This addendum fills the syllabus topics that the main notes covered only briefly. Every code block was run, and the printed output shown is the real output.

1 Data Structures and Sorting (Programming, DSA)

Linked lists

Ramu: Why not just use an array?
Somu: A linked list changes at the front in $O(1)$ by moving one pointer, but reaching the k-th item costs $O(k)$. Reversal flips each pointer once: $O(n)$.

```
class Node:
    def __init__(self, val, nxt=None):
        self.val, self.nxt = val, nxt      # value + pointer to next node
head = Node(1, Node(2, Node(3)))           # 1 -> 2 -> 3
def to_list(n):
    out = []
    while n: out.append(n.val); n = n.nxt  # walk the pointers: O(n)
    return out
def reverse(head):
    prev = None
    while head:
        nxt = head.nxt                     # remember the rest
        head.nxt = prev                    # flip the pointer
        prev, head = head, nxt             # step forward
    return prev
print(to_list(head), to_list(reverse(head)))
# UPDATE: insert at the head is O(1); finding the k-th item is O(k)
```

Output:

```
[1, 2, 3] [3, 2, 1]
```

Binary search trees

Smaller keys go left, bigger go right. Search cost equals the height: $O(\log n)$ balanced, $O(n)$ when keys arrive sorted.

```
class T:
    def __init__(self, k): self.k, self.l, self.r = k, None, None
def insert(t, k):
    if t is None: return T(k)
    if k < t.k: t.l = insert(t.l, k)       # smaller goes left
    else: t.r = insert(t.r, k)             # bigger goes right
    return t
def height(t): return 0 if t is None else 1 + max(height(t.l), height(t.r))
```

```
def inorder(t): return inorder(t.l) + [t.k] + inorder(t.r) if t else []
root = None
for k in [50, 30, 70, 20, 40, 60, 80]: root = insert(root, k)
print(inorder(root), height(root))         # in-order = sorted
skew = None
for k in [10, 20, 30, 40, 50, 60, 70]: skew = insert(skew, k)
print(height(skew))                        # sorted input = a chain
# UPDATE: search costs O(height): log n if balanced, n if skewed
```

Output:

```
[20, 30, 40, 50, 60, 70, 80] 3
7
```

Hash tables and collisions

A hash function maps a key to a slot. Different keys can share a slot (a collision); chaining keeps a list per slot. Load factor $\alpha = n/m$; the average lookup is $1 + \alpha$.

```
M = 5
table = [[] for _ in range(M)]             # each slot holds a chain
def put(k): table[k % M].append(k)         # hash = k mod 5
for k in [12, 22, 7, 17, 3, 27]: put(k)
print(table)
print(len(table[2]))                       # keys 12,22,7,17,27 share slot 2
print(6 / M)                               # load factor = n / slots
# UPDATE: average cost is 1 + load factor; a bad hash makes it O(n)
```

Output:

```
[[], [], [12, 22, 7, 17, 27], [3], []]
5
1.2
```

Insertion and selection sort

Insertion sort is $O(n)$ on sorted input and $O(n^2)$ on reversed input. Selection sort always does about $n^2/2$ comparisons but at most n swaps.

```
def isort(a):
    a, shifts = a[:], 0
    for i in range(1, len(a)):
        key, j = a[i], i - 1
        while j >= 0 and a[j] > key:       # shift bigger items right
            a[j + 1] = a[j]; j -= 1; shifts += 1
        a[j + 1] = key
    return a, shifts
def ssort(a):
    a, swaps = a[:], 0
    for i in range(len(a)):
        m = min(range(i, len(a)), key=lambda x: a[x])   # find the smallest
        a[i], a[m] = a[m], a[i]; swaps += 1
    return a, swaps
print(isort([1, 2, 3, 4, 5, 6])[1], isort([6, 5, 4, 3, 2, 1])[1])
print(ssort([5, 2, 4, 1, 3]))
# UPDATE: insertion is O(n) if sorted, O(n^2) if reversed
```

Output:

```
0 15
([1, 2, 3, 4, 5], 5)
```

Quick sort

Average $O(n \log n)$; with a first-item pivot on sorted input it degrades to $n(n-1)/2$ comparisons. A random pivot makes the bad case unlikely.

```
import random
def qsort(a, c):
    if len(a) <= 1: return a
    p = a[0]                                # pivot = first item
    lo = [x for x in a[1:] if x < p]
    hi = [x for x in a[1:] if x >= p]
    c[0] += len(a) - 1                      # comparisons this level
    return qsort(lo, c) + [p] + qsort(hi, c)
random.seed(2)
n = 200
c1, c2 = [0], [0]
qsort(random.sample(range(n), n), c1)
qsort(list(range(n)), c2)
print(c1[0], c2[0], n * (n - 1) // 2)      # random vs already sorted
# UPDATE: random pivot keeps the expected cost at n log n
```

Output:

```
1707 19900 19900
```

2 Probability and Statistics

Mean, median, mode, spread

The mean chases outliers, the median does not. Population standard deviation is the root of the average squared deviation.

```
data = [2, 4, 4, 4, 5, 5, 7, 9]
n = len(data)
mean = sum(data) / n
s = sorted(data)
median = (s[n//2 - 1] + s[n//2]) / 2       # middle of the sorted list
mode = max(set(data), key=data.count)      # most frequent value
var = sum((x - mean)**2 for x in data) / n # population variance
print(mean, median, mode, var ** 0.5)
big = data + [100]                         # add one outlier
bs = sorted(big)
print(round(sum(big)/len(big), 2), bs[len(big)//2])
# UPDATE: the mean chases outliers, the median does not
```

Output:

```
5.0 4.5 4 2.0
15.56 5
```

Covariance and correlation

$r = \mathrm{cov}(x, y)/(\sigma_x \sigma_y)$. $r = 0$ does not mean independent: $y = x^2$ has covariance 0.

```
xs = [1, 2, 3, 4, 5, 6]
ys = [2.1, 3.9, 6.2, 7.8, 10.1, 12.2]
n = len(xs)
mx, my = sum(xs)/n, sum(ys)/n
cov = sum((x-mx)*(y-my) for x, y in zip(xs, ys)) / n
sx = (sum((x-mx)**2 for x in xs) / n) ** 0.5
sy = (sum((y-my)**2 for y in ys) / n) ** 0.5
print(round(cov, 3), round(cov / (sx * sy), 4))   # r = cov / (sx sy)
zs = [-3, -2, -1, 0, 1, 2, 3]
zy = [z * z for z in zs]                  # y is fully determined by x
mz, mw = 0, sum(zy) / 7
print(sum((a - mz) * (b - mw) for a, b in zip(zs, zy)) / 7)
# UPDATE: r measures LINEAR link only
```

Output:

```
5.892 0.9991
0.0
```

The t test

$t = (\bar{x} - \mu_0)/(s/\sqrt{n})$ with s using $n - 1$. Compare $|t|$ with the critical value for $n - 1$ degrees of freedom (2.262 for 9 df at 5% two-sided).

```
data = [5.1, 4.9, 5.6, 5.8, 6.0, 5.5, 5.3, 6.1, 5.7, 5.9]
mu0 = 5.0                                 # claim: the mean is 5.0
n = len(data)
m = sum(data) / n
sd = (sum((x - m)**2 for x in data) / (n - 1)) ** 0.5   # sample sd, n-1
t = (m - mu0) / (sd / n ** 0.5)           # t statistic
crit = 2.262                              # t table, df=9, 5% two-sided
print(round(m, 3), round(sd, 3), round(t, 3))
print(abs(t) > crit)                      # True -> reject the claim
# UPDATE: use t (not z) when sigma is unknown and n is small
```

Output:

```
5.59 0.393 4.749
True
```

3 Machine Learning

Hierarchical clustering and linkage

Merge the closest clusters repeatedly. Single linkage uses the closest pair (chains), complete linkage the farthest pair (compact).

```
pts = [1, 2, 3, 5, 7, 8, 9]
cl = [[p] for p in pts]                   # start: every point alone
def d_single(a, b): return min(abs(x - y) for x in a for y in b)
def d_complete(a, b): return max(abs(x - y) for x in a for y in b)
def run(dist):
    c, merges = [x[:] for x in cl], []
```

```
    while len(c) > 1:
        i, j = min(((i, j) for i in range(len(c)) for j in range(i+1, len(c))),
                   key=lambda ij: dist(c[ij[0]], c[ij[1]]))
        merges.append((c[i] + c[j], dist(c[i], c[j])))   # merge the closest pair
        c = [x for k, x in enumerate(c) if k not in (i, j)] + [c[i] + c[j]]
    return merges
print(run(d_single)[-2:])
print(run(d_complete)[-2:])
# UPDATE: single linkage chains; complete linkage keeps clusters compact
```

Output:

```
[([5, 3, 1, 2], 2), ([9, 7, 8, 5, 3, 1, 2], 2)]
[([1, 2, 3, 5], 4), ([9, 7, 8, 1, 2, 3, 5], 8)]
```

Linear discriminant analysis

Fisher direction $w = S_w^{-1}(m_1 - m_0)$, supervised, assumes equal class covariance.

```
x0 = [(1, 2), (2, 3), (3, 3), (2, 1)]      # class 0
x1 = [(6, 5), (7, 7), (8, 6), (7, 4)]      # class 1
mean = lambda xs: [sum(c) / len(xs) for c in zip(*xs)]
m0, m1 = mean(x0), mean(x1)
def scatter(xs, m):
    s = [[0, 0], [0, 0]]
    for x in xs:
        d = [x[0] - m[0], x[1] - m[1]]
        for i in range(2):
            for j in range(2): s[i][j] += d[i] * d[j]
    return s
S0, S1 = scatter(x0, m0), scatter(x1, m1)
Sw = [[S0[i][j] + S1[i][j] for j in range(2)] for i in range(2)]  # within-class
det = Sw[0][0] * Sw[1][1] - Sw[0][1] * Sw[1][0]
inv = [[Sw[1][1] / det, -Sw[0][1] / det], [-Sw[1][0] / det, Sw[0][0] / det]]
dm = [m1[0] - m0[0], m1[1] - m0[1]]
w = [inv[0][0]*dm[0] + inv[0][1]*dm[1], inv[1][0]*dm[0] + inv[1][1]*dm[1]]
proj = lambda p: w[0] * p[0] + w[1] * p[1]  # project onto w
print([round(v, 3) for v in w])
print(round(proj(m0), 2), round(proj(m1), 2))
# UPDATE: Fisher LDA direction w = Sw^-1 (m1 - m0)
```

Output:

```
[1.194, 0.111]
2.64 8.97
```

Multiple linear regression

Normal equations $X^T X w = X^T y$. Strongly correlated features make $X^T X$ nearly singular; drop one or use ridge.

```
X = [[1, 1, 2], [1, 2, 1], [1, 3, 4], [1, 4, 3], [1, 5, 5]]   # 1, x1, x2
y = [3, 4, 9, 10, 14]
n = 3
A = [[sum(r[i]*r[j] for r in X) for j in range(n)] for i in range(n)]   # X^T X
b = [sum(r[i]*t for r, t in zip(X, y)) for i in range(n)]               # X^T y
for i in range(n):                          # Gaussian elimination
    p = A[i][i]
```

```
    A[i] = [v / p for v in A[i]]; b[i] /= p
    for k in range(n):
        if k != i:
            f = A[k][i]
            A[k] = [a - f * c for a, c in zip(A[k], A[i])]; b[k] -= f * b[i]
print([round(v, 2) for v in b])            # intercept, w1, w2
# UPDATE: two features that move together make X^T X nearly singular
```

Output:

```
[-1.0, 2.0, 1.0]
```

4 Artificial Intelligence

Iterative deepening

Depth-limited DFS with limits $0, 1, 2, \ldots$: memory $O(bd)$ like DFS, shallowest goal like BFS.

```
g = {"A": ["B", "C"], "B": ["D", "E"], "C": ["F", "G"], "D": [], "E": [], "F": [], "G": []}
def dls(u, goal, limit, seen):             # depth-limited DFS
    seen.append(u)
    if u == goal: return True
    if limit == 0: return False
    return any(dls(v, goal, limit - 1, seen) for v in g[u])
total = 0
for depth in range(4):                     # deepen one level at a time
    seen = []
    found = dls("A", "G", depth, seen)
    total += len(seen)
    print(depth, found, len(seen))
    if found: break
print(total)
# UPDATE: memory like DFS (O(bd)), answer like BFS (shallowest)
```

Output:

```
0 False 1
1 False 3
2 True 7
11
```

Conditional independence and d-separation

Observing the middle node blocks a chain or fork but opens a collider (explaining away).

```
from itertools import product
def indep(joint, given):                   # is A independent of C (given B?)
    for g in ([0, 1] if given else [None]):
        keep = {s: p for s, p in joint.items() if g is None or s[1] == g}
        z = sum(keep.values())
        pa = {a: sum(p for s, p in keep.items() if s[0] == a) / z for a in (0, 1)}
        pc = {c: sum(p for s, p in keep.items() if s[2] == c) / z for c in (0, 1)}
        for a, c in product((0, 1), repeat=2):
            pac = sum(p for s, p in keep.items() if s[0] == a and s[2] == c) / z
            if abs(pac - pa[a] * pc[c]) > 1e-9: return False
    return True
S = list(product((0, 1), repeat=3))        # state = (A, B, C)
```

```
chain = {s: .5 * (.9 if s[1] == s[0] else .1) * (.8 if s[2] == s[1] else .2) for s in S}
coll = {s: .25 * (.9 if s[1] == (s[0] | s[2]) else .1) for s in S}
print(indep(chain, False), indep(chain, True))   # chain A -> B -> C
print(indep(coll, False), indep(coll, True))     # collider A -> B <- C
# UPDATE: observing the middle blocks a chain but OPENS a collider
```

Output:

```
False True
True False
```

5 Database Management

Relational algebra

σ selects rows, π projects columns, join combines on keys, division finds "for all" matches; the SQL form uses GROUP BY and HAVING.

```
import sqlite3
S = {("s1", "ann", "cs"), ("s2", "bob", "ee"), ("s3", "cy", "cs")}      # Student(id,name,dept)
E = {("s1", "db"), ("s1", "ml"), ("s2", "db"), ("s3", "db"), ("s3", "ml")}  # Enrolled(id,course)
sel = {r for r in S if r[2] == "cs"}                 # sigma: pick rows
proj = {(r[1],) for r in S}                          # pi: pick columns
join = {(a[0], a[1], b[1]) for a in S for b in E if a[0] == b[0]}   # natural join
print(sorted(sel), len(join))
need = {"db", "ml"}                                  # division: enrolled in ALL of need
div = {s for s, _ in E if need <= {c for t, c in E if t == s}}
print(sorted(div))
db = sqlite3.connect(":memory:")
db.execute("create table e(id, course)")
db.executemany("insert into e values (?, ?)", sorted(E))
q = "select id from e where course in ('db','ml') group by id having count(distinct course) = 2"
print(sorted(r[0] for r in db.execute(q)))           # same answer in SQL
# UPDATE: relational algebra uses SETS; SQL tables are BAGS
```

Output:

```
[('s1', 'ann', 'cs'), ('s3', 'cy', 'cs')] 5
['s1', 's3']
['s1', 's3']
```

Sets versus bags

Algebra works on sets (no duplicates); SQL tables are bags, so use DISTINCT to match π.

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("create table s(id, dept)")
db.executemany("insert into s values (?, ?)", [(1, "cs"), (2, "cs"), (3, "ee")])
print(db.execute("select dept from s").fetchall())           # SQL keeps duplicates
print(db.execute("select distinct dept from s").fetchall())  # pi removes them
print(len({"cs", "cs", "ee"}))                               # a set has no duplicates
# UPDATE: add DISTINCT to match the algebra
```

Output:

```
[('cs',), ('cs',), ('ee',)]
[('cs',), ('ee',)]
2
```